

Project AmarketBoudingBoxes

Dataset YOLO automático desde Amarket

Guillermo Carvajal Vaca Monserrat Barba Pablo Linares Andrés Poiche

Visión Computacional
Entrega: 19 de agosto de 2026

Resumen

Este trabajo construye un *pipeline* automático de anotación monoclasa (0: **product**) a partir del catálogo público de Amarket: 655 imágenes aceptadas, splits 459/98/98 sin fuga de SKU, y un detector YOLO11n (Experimento A) evaluado sobre prueba bloqueada. Un replanteo del alcance, comunicado por el docente durante el desarrollo, exige además identificar *qué* producto aparece en escenas multiproducto tipo caja de supermercado. Para responder sin invalidar el contrato monoclasa congelado (`class_id=0` en todo el dataset; SKU y nombre nunca como clase, dado que el catálogo tiene aproximadamente una muestra por SKU), se documenta una decisión de alcance no congelada (ADR-006) y se implementa una arquitectura de dos etapas: el mismo detector monoclasa localiza cada objeto, y una segunda etapa de *retrieval* visual (SIFT contra un catálogo de 655 referencias) resuelve la identidad sin reentrenar el detector ni clasificar por SKU. El detector, aumentado con 197 escenas sintéticas multiproducto (de una meta de 655, limitación declarada), alcanza $mAP@50-95=0.9715$ sobre prueba real bloqueada. La identificación, evaluada de forma cerrada sobre 481 placements sintéticos, alcanza 84.62 % top-1 y 92.31 % top-5, con 0 % de tasa UNKNOWN. Se reportan también los errores de identidad, concentrados en variantes de una misma línea de producto y envases de baja textura.

1. Introducción

El etiquetado manual de *bouding boxes* no escala. Anotar a mano cada producto de un catálogo comercial exige un esfuerzo proporcional al número de imágenes, es costoso de verificar y difícil de reproducir: dos anotadores distintos producen cajas distintas sobre la misma fotografía.

Este trabajo construye un *pipeline* automático de anotación a partir de un catálogo real. Las fotografías de producto de un comercio electrónico presentan una regularidad aprovechable: muestran un único artículo aislado sobre fondo uniforme o transparente. Esa regularidad permite derivar la caja delimitadora de forma determinista a partir de los píxeles extremos del producto, sin intervención humana y sin ambigüedad entre anotadores.

El objetivo base es un dataset de detección **monoclasa**. La única clase YOLO es 0: **product**. El SKU, el nombre y la descripción se conservan como metadatos de procedencia y trazabilidad, no como clases YOLO: el detector resuelve *dónde* está el producto, no *cuál* es.

Durante el desarrollo, el docente replanteó el objetivo funcional final para simular una caja de supermercado: dada una escena con varios productos, el sistema debe localizar cada artículo *e* identificar cuál es. Convertir `class_id` en un identificador de SKU es estadísticamente inviable —el catálogo aporta aproximadamente una muestra por SKU, insuficiente para construir train/val/test por clase— y además violaría el contrato monoclasa congelado del dataset base. La respuesta documentada (Anexo/ADR-006, no congelado) es una arquitectura de **dos etapas**: el detector monoclasa localiza, y una segunda etapa de *retrieval* visual identifica contra un catálogo de referencia, sin reentrenar ni reclasificar el detector.

2. Marco teórico

2.1. Detección de objetos y formato YOLO

YOLO codifica cada caja como `<class_id><x_c><y_c><w><h>`, valores normalizados a $[0, 1]$ (independientes de la resolución); en el dataset monoclasa, `class_id` es siempre 0. Dada una imagen $W \times H$ y los

píxeles extremos inclusivos $x_{\min}, x_{\max}, y_{\min}, y_{\max}$ del producto, con borde semiabierto ($x_{\max} + 1, y_{\max} + 1$, evitando el truncamiento de un píxel):

$$x_c = \frac{x_{\min} + x_{\max} + 1}{2W}, \quad y_c = \frac{y_{\min} + y_{\max} + 1}{2H}, \quad w = \frac{x_{\max} - x_{\min} + 1}{W}, \quad h = \frac{y_{\max} - y_{\min} + 1}{H} \quad (1)$$

Los cuatro valores se escriben con seis decimales, precedidos de `class_id=0`.

2.2. Datasets de retail densamente anotados

SKU-110K [1] (11.762 imágenes, $\sim 1.73\text{M}$ de cajas) anota estantería densa como un **único tipo de objeto**: localización en escenas densas, no identificación (el «110K» refiere a la diversidad de SKU en escena, no a clases anotadas). **MVTec D2S** [3] aporta segmentación de instancias con separación deliberada train/eval. **RP2K** [2] aborda reconocimiento *fine-grained* entre artículos. La diferencia con este trabajo es de escena y objetivo: los tres operan sobre estantería en tienda; aquí se parte de catálogo con un producto aislado sobre fondo controlado, lo que habilita la caja determinista, y la identificación (cuando se requiere) se resuelve aparte, por *retrieval*, no por clases.

3. Metodología

3.1. Visión general del *pipeline*

Seis etapas con propietario y contrato definidos: (1) **adquisición** (`src/scrapper_extraction.py` descarga el catálogo público con procedencia trazable); (2) **validación de fondo** (producto aislado, fondo uniforme o transparente; rechazos registrados con motivo); (3) **píxeles extremos** sobre la máscara del producto; (4) **conversión a YOLO**; (5) **particionado** 70/15/15 sin fuga; (6) **modelado y evaluación** (*fine-tuning*, evaluación e inferencia en CPU).

3.2. Particionado, entrenamiento y evaluación

El dataset se divide 70/15/15 por *grupos* (SKU, hash de bytes, grupo de duplicados o linaje), no por imagen suelta, para que ninguna partición filtre al resto y las métricas no queden infladas por memorización. Se parte de un detector preentrenado (YOLO11n), ajustado sobre entrenamiento y seleccionado sobre validación; semilla fija 42, hiperparámetros en configuración versionada, nunca embebidos en código. Prueba permanece bloqueada durante todo el ajuste y se consulta una sola vez, sin realimentar ninguna decisión de diseño. La inferencia final se verifica también en CPU, sin CUDA.

3.3. Aumentación sintética y retrieval de identidad

Experimento B añade escenas sintéticas multiproducto: composiciones deterministas de cutouts *train-only*, con geometría de oclusión controlada y verificación de hash. Meta contractual: 655 escenas; la corrida completa se detuvo en SYN_0198 al agotar el presupuesto de búsqueda estructurado *respetando* el límite de oclusión, y se congelan las 197 escenas válidas (limitación declarada), sin tocar validación/prueba (98/98 reales).

La segunda etapa recibe el *crop* de cada caja YOLO y lo compara, vía descriptores SIFT y *matching* por fuerza bruta con prueba de razón de Lowe (0.75), contra un catálogo de 655 referencias (cutout de entrenamiento o foto real), indexadas una sola vez. Se devuelve el SKU con más coincidencias válidas; por debajo de un umbral mínimo se declara UNKNOWN. Determinista, sin entrenamiento ni pesos adicionales: SIFT/ORB ya están disponibles en el entorno; se descartó un backend de *embeddings* preentrenados por exigir descargar pesos ausentes localmente.

4. Resultados

4.1. Detección: Experimento A vs. Experimento B

Experimento A: 459 imágenes reales, un producto por imagen, YOLO11n, 40/40 épocas, CPU (peso sha256:9149dfef..., evidencia en docs/evidence/G4_STATUS.md). Experimento B: 656 imágenes (459

reales + 197 sintéticas multiproducto), dos configuraciones GPU idénticas salvo `imgsz` (B0=416, T1=512); T1 se selecciona por mayor `mAP@50-95` en validación (B0: P=0.998, R=0.990, `mAP@50`=0.995, `mAP@50-95`=0.987, no seleccionado). T2 (`batch=16`) queda deliberadamente diferido: B0 frente a T1 ya constituye el *tuning* comparativo requerido, y priorizar la identificación de producto era el objetivo funcional replanteado.

Tabla 1: Métricas de detección, una sola evaluación de prueba por experimento.

Métrica	A – Val	A – Prueba	B/T1 – Val	B/T1 – Prueba
Precisión	0.999	1.000000	0.999	0.985658
Recall	0.990	1.000000	1.000	1.000000
F1	—	1.000000	—	0.992777
<code>mAP@50</code>	0.995	0.995000	0.995	0.994091
<code>mAP@50-95</code>	0.986	0.992875	0.991	0.971511

4.2. Identificación de producto (segunda etapa)

Evaluación end-to-end sobre las 197 escenas sintéticas / 481 *placements* reales, emparejando cada detección con su verdad de composición por IoU (≥ 0.5).

Tabla 2: Identificación de producto sobre 481 placements.

Métrica	Valor
Detecciones totales (GT=481)	482
Detector recall (IoU ≥ 0.5)	1.000000
Detector precisión (IoU ≥ 0.5)	0.997925
Falsos positivos	1
Identidad top-1	407/481 = 84.6154 %
Identidad top-5	444/481 = 92.3077 %
Exactitud end-to-end (IoU $\wedge$ SKU top-1)	407/481 = 84.6154 %
Tasa UNKNOWN	0.000000

Advertencia de dominio cerrado: las escenas sintéticas de evaluación derivan de los mismos *coutouts* que forman el catálogo de referencia. Esta cifra mide si el *retrieval* encuentra su propia fuente, no generalización a fotografías nuevas de una caja real.

4.3. Curvas de entrenamiento

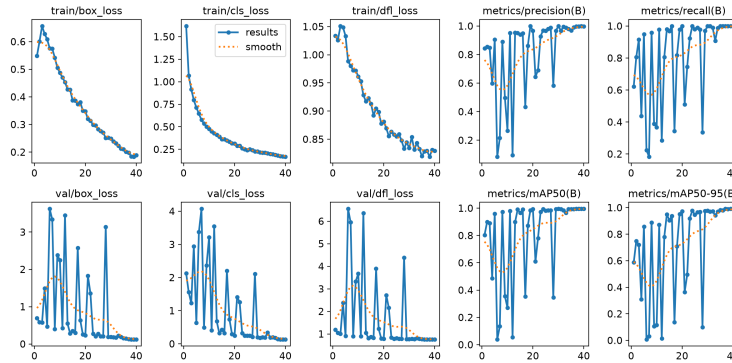


Figura 1: Curvas de entrenamiento de T1 (Experimento B, ganador): pérdidas, precisión, recall y mAP por época.

Diez aciertos y diez errores de identificación, con caja YOLO, nombre predicho/esperado y una hipótesis breve por error, se presentan en el Apéndice A para no extender el cuerpo principal.

5. Discusión

La aumentación sintética multiproducto mejora mAP@50-95 en validación (0.987→0.991) y el detector T1 mantiene un desempeño casi perfecto sobre prueba real bloqueada (0.9715), pese a entrenar con solo 197 de las 655 escenas planificadas. La tarea de localización monoclasa parece saturada para este dominio: producto aislado sobre fondo controlado, una sola clase, sin ambigüedad de forma.

La identificación es la tarea genuinamente difícil. El 84.6 % de exactitud top-1 no es una cifra intercambiable con las métricas de detección: mide un problema distinto (identidad de instancia por *retrieval*, no localización) y, como se advierte arriba, en un dominio cerrado. Los errores observados (figura 3) no son aleatorios: se concentran en (a) variantes de una misma línea de producto, donde el empaque difiere solo en texto o color pequeño (mismo caso que llevó a descartar `class_id` por SKU en primer lugar), y (b) envases lisos con poca textura local, donde SIFT tiene pocos puntos de interés que extraer. Ambos son límites conocidos y documentados de los descriptores locales clásicos, no una falla de implementación: la verificación cruzada entre el subconjunto de 20 escenas usado para elegir el *backend* y la evaluación completa de 481 *placements* reprodujo exactamente los mismos resultados fila por fila cuando se usó el mismo peso, lo que descarta una discrepancia de código.

6. Limitaciones

La derivación determinista de cajas por píxeles extremos sigue dependiendo de que la fotografía de *catálogo* muestre un único producto sobre fondo uniforme o transparente; esa etapa no cambia. La novedad de este trabajo —la composición sintética y la identificación— sí opera sobre escenas multiproducto, pero con límites propios:

- **Dataset sintético incompleto (197/655).** La corrida completa falló en SYN_0198 por agotamiento del presupuesto de búsqueda estructurado, respetando el límite de oclusión en vez de violarlo. Las 197 escenas usadas son todas de dificultad *basic* (2–3 objetos); el detector no fue expuesto durante el entrenamiento a las densidades *medium/hard/extreme* planificadas.
- **Evaluación de identidad cerrada.** El 84.6 % top-1 mide *retrieval* contra el catálogo del que las escenas derivan, no generalización a fotografías nuevas de una caja real.
- **SIFT confunde variantes de línea y envases lisos,** como se discute arriba; no hay mecanismo de *fine-tuning* sobre esos casos difíciles en este trabajo.
- El dataset procede de un único comercio, por lo que su diversidad visual está acotada por el catálogo de origen y sus convenciones fotográficas.

7. Conclusiones

El objetivo de localización monoclasa se cumple con alta fiabilidad tanto en el dominio de producto único (Experimento A, mAP@50-95 =0.9929 en prueba) como en el multiproducto aumentado sintéticamente (Experimento B/T1, mAP@50-95 =0.9715 en prueba), sin necesidad de convertir `class_id` en un identificador de producto —decisión que además habría sido inviable dado el catálogo disponible (aproximadamente una muestra por SKU). La arquitectura de dos etapas responde al replanteo docente preservando el contrato monoclasa congelado: localizar y identificar son problemas distintos, resueltos por componentes distintos, con sus propias métricas y sus propias limitaciones. La identificación por *retrieval* visual es funcional (84.6 % top-1, 92.3 % top-5) pero no está lista para generalizar fuera del catálogo cerrado evaluado; una arquitectura de *embeddings* aprendidos, hoy descartada por no haber pesos preentrenados disponibles localmente, es la vía natural para mejorarla.

8. Reproducibilidad

8.1. Entorno

Experimento A: Python 3.11.9, dependencias fijadas en `requirements.txt`, CPU. Experimento B (GPU): entorno `ComputerVision` (WSL2 Ubuntu), Python 3.12.13, PyTorch 2.13.0+cu126, Ultralytics 8.4.120, CU-

DA 12.6, NVIDIA GeForce RTX 4050 Laptop (6GB). La identificación usa OpenCV 5.0.0 (SIFT_create, BFMatcher), ya presente en el entorno GPU sin instalación adicional.

8.2. Comandos

El flujo completo se ejecuta desde la línea de órdenes, sin depender de *notebooks* (el *notebook* de CPU es solo demostración final):

```
# Experimento A (CPU)
python src/train.py      --config configs/baseline.yaml --data configs/dataset.yaml \
                        --output-dir outputs/runs/baseline --seed 42
python src/evaluate.py --model <peso> --data configs/dataset.yaml --split test \
                        --allow-test --output-dir outputs/evaluation/baseline

# Experimento B (GPU): sintetico (ADR-005) + entrenamiento/eval/identificacion (ADR-006)
python -m src.data.assign_synthetic_sources --scene-plan <plan> \
      --cutout-manifest <manifest> --output <assignments.csv>
python -m src.data.make_synthetic_scenes --assignments <assignments.csv> \
      --cutout-root <cutouts> --output-root <synthetic> --manifest <scene_manifest.csv> \
      --cutout-manifest <manifest> --generator-commit <sha>
python src/train.py      --config configs/experiment_b_gpu_t1.yaml \
                        --data <dataset_wsl.yaml> --output-dir <run_t1> --seed 42
python src/evaluate.py --model <run_t1>/weights/best.pt --data <dataset_wsl.yaml> \
                        --split test --allow-test --output-dir <test_eval_t1>
python src/predict.py   --model <run_t1>/weights/best.pt --input <escena.jpg> \
                        --catalog catalog_identity.csv --output-dir <resultados>
```

8.3. Determinismo

La semilla global es 42. Cada ejecución registra la configuración resuelta, la versión del entorno y el hash del peso empleado, de modo que una evaluación repetida sobre el mismo peso produce métricas idénticas.

Referencias

- [1] E. Goldman, R. Herzig, A. Eisenschlat, J. Goldberger y T. Hassner. *Precise Detection in Densely Packed Scenes*. CVPR, 2019. https://openaccess.thecvf.com/content_CVPR_2019/html/Goldman_Precise_Detection_in_Densely_Packed_Scenes_CVPR_2019_paper.html
- [2] J. Peng, C. Xiao y Y. Li. *RP2K: A Large-Scale Retail Product Dataset for Fine-Grained Image Classification*. arXiv:2006.12634, 2020. <https://arxiv.org/abs/2006.12634>
- [3] MVTec Software GmbH. *MVTec D2S: Densely Segmented Supermarket Dataset*. <https://www.mvtec.com/company/research/datasets/mvtec-d2s>

A. Ejemplos de identificación (10 aciertos, 10 errores)

Seleccionados directamente de `end_to_end_detail.csv` en orden de archivo (sin selección favorable), detector T1 + *retrieval* SIFT. Cada imagen muestra la caja YOLO, el nombre predicho y la confianza de detección; en los errores se indica también el nombre esperado. Verificación automática: la predicción/confianza dibujada en cada imagen coincide exactamente con su fila de `end_to_end_detail.csv` (0 discrepancias sobre 20 filas recomputadas con el mismo peso, catálogo y método de *crop* que la evaluación gobernante).



Figura 2: Diez identificaciones correctas.



Figura 3: Diez identificaciones incorrectas: nombre predicho vs. esperado. La caja YOLO es correcta en los diez casos ($\text{IoU} \geq 0.94$); el error es siempre de identidad, nunca de localización.

Tabla 3: Hipótesis de causa por error (ver también §5, Discusión).

Escena	Esperado → Predicho	Hipótesis
SYN_0001#0	Serum Izeze Hyper Cica → Crema Facial Izeze Hyper Cica	Misma línea de producto, envase casi idéntico
SYN_0001#1	Labial Maybelline tono 505 → tono 520	Misma línea, solo cambia el tono (acierta en top-5)
SYN_0002#1	Aceite de Oliva 250 Ml → 500 Ml	Mismo producto, tamaño de envase distinto
SYN_0003#0	Desodorante Dove → Vape Waka	Similitud baja (79); crop con poca textura distintiva
SYN_0003#1	Helado Delizia → Regaliz Dori	Similitud baja (76), <i>matching</i> ruidoso
SYN_0004#0	Tónico Izeze → Crema Facial Izeze	Misma línea de producto
SYN_0005#0	Desodorante Dove Serum → Nosotras Rapigel	Similitud baja (57)
SYN_0005#1	Sombra Resist Coco → Jarra Stanley	Similitud casi en el umbral UNKNOWN (9); crop pequeño
SYN_0006#0	Protector Solar con color → sin color	Mismo producto, variante distinta
SYN_0007#0	Acondicionador Dove → Stanley Quencher	Botella lisa, pocos puntos de interés para SIFT